

Understanding the Convex Triangular Shape Model

Illumination, visibility, light-curve synthesis, and the supplied Juno code

A practical study guide based on Humes and Agarwal (2026)

September 2026

Contents

1. What the shape model represents	3
2. What “convex” means	4
3. How an OBJ file builds the surface	4
3.1 Vertex records	4
3.2 Face records	4
3.3 How thousands of triangles become one body	4
4. What is computed for every triangle	5
4.1 Two edge vectors	5
4.2 Cross product	6
4.3 Facet area	6
4.4 Unit surface normal	6
4.5 Facet centroid	6
5. The coordinate frames	7
5.1 Ecliptic frame	7
5.2 Asteroid-fixed frame	7
6. What <code>kete_ephem()</code> does	7
6.1 Why the vectors are negated later	7
6.2 Light-time correction	8
7. What <code>asterocentric()</code> does	8
8. Illumination test: is sunlight reaching this facet?	9
8.1 Where the illumination test is used in the code	9
8.2 Small numerical example	10
9. Visibility test: can the observer see this facet?	10
10. Why both tests are necessary	10
11. Why dot products work only because the model is convex	11
12. The scattering law: how much light comes from an accepted facet?	12
12.1 Example	12
13. Area and reflectance weighting	12
13.1 Example of a synthetic red spot	13
14. Summing the unresolved asteroid	13

15. Why the code appends [1, 0, 0]	14
16. Flux-to-magnitude conversion	14
16.1 Distance correction	14
16.2 Phase function	15
17. Array dimensions inside observation()	15
18. Where this code belongs in your project	15
Stage A - Predict geometry	15
Stage B - Predict uniform-surface brightness	16
Stage C - Compare real photometry with the model	16
Stage D - Apply the colour-variation statistic	16
19. What the two major code sections are for	16
20. What the supplied demonstration currently does	17
21. Exact walkthrough of spot_detection()	17
21.1 Residual vector	17
21.2 Average illumination-viewing direction	17
21.3 Regression direction	17
21.4 Spearman correlation in each band	18
21.5 Fisher difference score	18
22. Important distinction: illumination test versus statistical test	18
23. Issues to correct before running the supplied code	18
23.1 Missing stats import	18
23.2 Undefined plot	19
23.3 Working directory for juno.obj	19
23.4 Required packages	19
23.5 Inputs must be aligned	19
23.6 Two unused arrays	19
24. How to use the code for one real ZTF asteroid	19
25. How a colour spot should enter a simulation	20
26. Validation tests you should perform first	20
26.1 Normal-length test	20
26.2 Outward-normal test	20
26.3 Closed-mesh test	20
26.4 Opposition symmetry test	21
26.5 Reversal test	21
26.6 Rotation periodicity test	21
26.7 Sphere test	21
26.8 Scale test	21
27. What can create a false colour signal	21
28. The complete scientific chain in one place	21
29. Plain-language explanation for a professor	22
30. Formula reference	22
References and software	23

This report explains what the convex triangular asteroid model is, how it is built, what the

illumination and visibility tests mean, where they appear in the supplied Python code, and how the code turns a three-dimensional model into a predicted light curve. It is written as a bridge between the Humes and Agarwal method and an actual DAMIT model.

The two supplied files are:

- `juno.obj`: the three-dimensional convex shape model of asteroid (3) Juno;
- `example_code.py`: functions for reading the model, computing observing geometry, generating synthetic photometry, and calculating the surface-colour test statistic.

The central idea is simple:

Divide the asteroid surface into many small triangles, calculate how much light every triangle sends toward Earth, and add those contributions.

The difficulty lies in calculating the orientation, illumination, visibility, scattering, distance and rotation correctly. This guide separates those pieces.

1. What the shape model represents

A telescope normally sees an asteroid as one unresolved point. It does not directly photograph its surface. Nevertheless, an asteroid's changing brightness contains information about its shape and rotation.

A shape model is a mathematical approximation of the asteroid's exterior. In this project, the exterior is a polyhedron made from flat triangular surface pieces called **facets**.

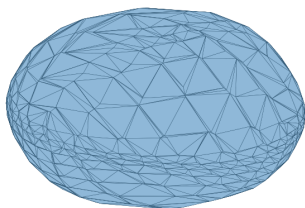
The supplied model contains:

Quantity	Value
Vertices	1,020
Triangular facets	2,036
Unique edges	3,054
Euler check, $V - E + F$	2

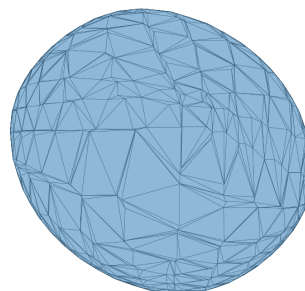
The value $V - E + F = 2$ is the expected Euler characteristic for a closed surface with sphere-like topology. All facets in the supplied file are also consistently wound so that their calculated normals point outward.

The supplied Juno OBJ model: 1,020 vertices joined into 2,036 triangular facets

Oblique view



Pole-on view



The triangles are not observations made by ZTF. They are the **physical model** used to predict what ZTF or another telescope should measure.

2. What “convex” means

A body is convex when the straight line joining any two points inside it remains inside it. Informally, it has no deep inward cavities, overhangs or holes.

A potato-like object can be convex even though it is irregular. A bowl, deep crater or dumbbell-shaped body can be non-convex.

Convexity produces an important computational shortcut:

On a convex closed body, an outward-facing facet is visible, and a Sun-facing facet is illuminated.

Therefore, two dot products are sufficient. A ray tracer is not needed.

For a non-convex body, a crater wall can block sunlight from a facet even when its normal points generally toward the Sun. Another part of the asteroid can also block the observer's line of sight. Those are self-shadowing and self-occlusion. The supplied code does not test for them.

3. How an OBJ file builds the surface

An OBJ file is a text description of geometry. Two record types matter here.

3.1 Vertex records

A line beginning with `v` defines a point in three-dimensional asteroid-fixed coordinates:

```
v 44.019230 23.254787 101.506337
```

This means

$$\mathbf{v}_1 = (44.019230, 23.254787, 101.506337).$$

The three numbers are coordinates, not right ascension, declination or geographic longitude and latitude. Their absolute scale does not affect a normalized light curve when the same model supplies both numerator and reference flux. Their ratios define the shape.

3.2 Face records

A line beginning with `f` selects three vertices and joins them into a triangular facet:

```
f 1017 1004 1003
```

This triangle uses OBJ vertices 1017, 1004 and 1003. OBJ numbering begins at 1, while NumPy arrays begin at 0. The importer therefore subtracts one.

```
tri = np.transpose([i1s - 1, i2s - 1, i3s - 1])
```

After this conversion, each row of triangles contains three valid NumPy indices.

3.3 How thousands of triangles become one body

Neighbouring faces reuse vertices and edges. Because every surface edge is shared by two facets, the collection forms a closed shell rather than a disconnected cloud of triangles.

Smaller and more numerous facets can represent shape details more accurately. The cost is that every observation requires calculations for more facets.

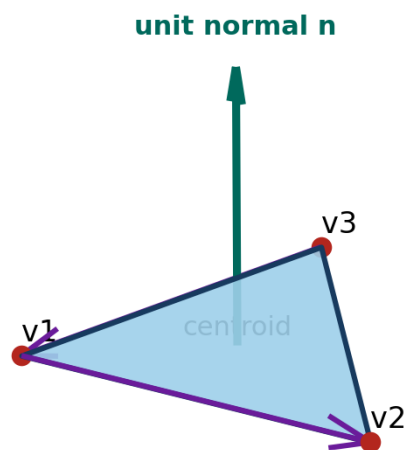
4. What is computed for every triangle

Suppose facet i has vertices

$$\mathbf{v}_{i1}, \quad \mathbf{v}_{i2}, \quad \mathbf{v}_{i3}.$$

The code obtains the corresponding rows in `coords1`, `coords2` and `coords3`.

What the code computes for every triangular facet



4.1 Two edge vectors

Two sides of the triangle can be written as

$$\mathbf{a}_i = \mathbf{v}_{i1} - \mathbf{v}_{i3},$$

$$\mathbf{b}_i = \mathbf{v}_{i2} - \mathbf{v}_{i1}.$$

The code uses exactly these differences:

```
self.cross_faces = np.cross(
    self.coords1 - self.coords3,
    self.coords2 - self.coords1
)
```

4.2 Cross product

The cross product

$$\mathbf{c}_i = \mathbf{a}_i \times \mathbf{b}_i$$

is perpendicular to the triangular plane. Its magnitude is

$$\|\mathbf{c}_i\| = \|\mathbf{a}_i\| \|\mathbf{b}_i\| \sin \gamma,$$

where γ is the angle between the chosen edges.

This magnitude equals the area of the parallelogram made by the two edge vectors.

4.3 Facet area

A triangle is half of that parallelogram, so

$$A_i = \frac{1}{2} \|\mathbf{c}_i\|.$$

The code is:

```
self.areas = np.sqrt(np.sum(self.cross_faces**2, axis=1)) / 2
```

Area matters because a large illuminated facet reflects more light than a small facet at the same orientation.

4.4 Unit surface normal

The unit normal is

$$\hat{\mathbf{n}}_i = \frac{\mathbf{c}_i}{\|\mathbf{c}_i\|} = \frac{\mathbf{c}_i}{2A_i}.$$

The code writes the same expression as

```
self.normals = 0.5 * self.cross_faces / [areas, areas, areas]
```

The unit normal has length one and points perpendicular to the face. It answers the question: **which way is this patch of surface facing?**

The ordering of the vertices controls whether the normal points outward or inward. Reversing two vertices reverses the cross product. The supplied Juno mesh passes an outward-normal check for all 2,036 facets.

4.5 Facet centroid

The approximate centre of the triangle is

$$\mathbf{q}_i = \frac{\mathbf{v}_{i1} + \mathbf{v}_{i2} + \mathbf{v}_{i3}}{3}.$$

The code stores its x , y and z components in `meanx`, `meany` and `meanz`. These centroids are useful for plotting a reflectance map or assigning a spot to a surface region. They are not used later by the `current_observation()` function.

5. The coordinate frames

Most confusion in this code comes from mixing two different frames.

5.1 Ecliptic frame

`kete_ephem()` initially computes vectors in a Solar System ecliptic coordinate system. In this frame, the axes do not rotate with Juno.

5.2 Asteroid-fixed frame

The triangular mesh is fixed to Juno. Its x , y and z axes rotate with the asteroid. The facet normals are stored in this frame.

The Sun and observer vectors must therefore be rotated into the same asteroid-fixed frame before any dot products are meaningful.

You cannot correctly compute

$$\hat{\mathbf{n}}_i \cdot \hat{\mathbf{s}}$$

if $\hat{\mathbf{n}}_i$ is asteroid-fixed but $\hat{\mathbf{s}}$ is still ecliptic. Both vectors must use the same axes.

6. What `kete_ephem()` does

The function `kete_ephem(times, site, ...)` calculates where the asteroid, Sun and observer are at every requested epoch.

Its output includes:

Column	Meaning
jd	Julian date
phase	Sun-asteroid-observer phase angle
r	Sun-asteroid distance in au
delta	observer-asteroid distance in au
sun2obj_*	vector from Sun to asteroid
obs2obj_*	vector from observer to asteroid
RA, Dec	apparent equatorial sky direction
elongation	angular distance from the Sun in the observer's sky

6.1 Why the vectors are negated later

`sun2obj` points from the Sun to the asteroid. At the asteroid, the required direction points from the asteroid back toward the Sun. Therefore,

$$\mathbf{s}_{\text{ast} \rightarrow \text{Sun}} = -\mathbf{r}_{\text{Sun} \rightarrow \text{ast}}$$

Similarly,

$$\mathbf{o}_{\text{ast} \rightarrow \text{observer}} = -\mathbf{r}_{\text{observer} \rightarrow \text{ast}}$$

That explains the minus signs inside `asterocentric()`.

6.2 Light-time correction

The image timestamp records when the photon arrived at the telescope. The visible facet was determined when that photon left the asteroid.

For distance Δ , the approximate travel time is

$$\tau = \frac{\Delta}{c}.$$

The code uses

$$c = 173.14463 \text{ au day}^{-1}.$$

Therefore, the rotational phase should be evaluated near

$$t_{\text{emit}} = t_{\text{receive}} - \frac{\Delta}{c}.$$

This matters because a main-belt asteroid can rotate noticeably during an approximately hour-long light travel time.

7. What `asterocentric()` does

This function rotates the Sun and observer directions from ecliptic coordinates into the coordinate system of the Juno mesh.

Its rotation tuple is

$$(\lambda, \beta, T, t_0, \phi_0),$$

where:

- λ : ecliptic longitude of the spin pole;
- β : ecliptic latitude of the spin pole;
- T : rotation period in days;
- t_0 : reference Julian date;
- ϕ_0 : rotation angle at the reference epoch.

For the supplied Juno example:

```
juno_rotation = (103, 27, 7.209531/24, 2434752.77932, 0)
```

Thus:

$$\lambda = 103^\circ, \quad \beta = 27^\circ,$$

and

$$T = \frac{7.209531}{24} \text{ days} \approx 0.300397 \text{ days}.$$

The rotation angle accumulated after the reference time is approximately

$$\theta(t) = 2\pi \frac{t - \tau - t_0}{T}.$$

The sequence `r1`, `r2`, `r3_obs`, `r4` aligns the pole, advances the rotational phase, and applies the reference angle. The outputs `new_sun` and `new_obs` are unit directions in Juno's rotating frame.

8. Illumination test: is sunlight reaching this facet?

Let:

- $\hat{\mathbf{n}}_i$ be the outward unit normal of facet i ;
- $\hat{\mathbf{s}}(t)$ be the unit direction from the asteroid toward the Sun at time t .

Define the incidence cosine

$$\mu_{0,i}(t) = \hat{\mathbf{n}}_i \cdot \hat{\mathbf{s}}(t).$$

Because both vectors have unit length,

$$\mu_{0,i} = \cos i_i,$$

where i_i is the incidence angle between the normal and the Sun direction.

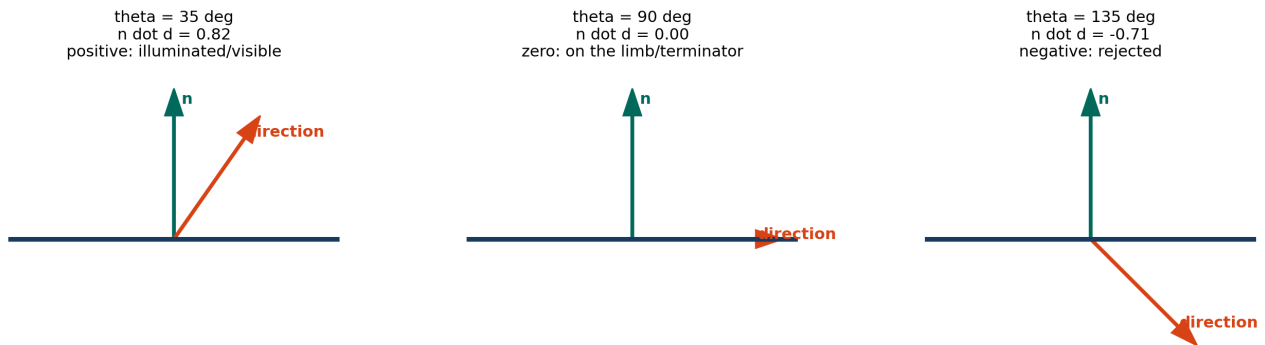
The illumination decision is

$$\text{facet is illuminated} \iff \mu_{0,i} > 0.$$

Interpretation:

- $\mu_{0,i} = 1$: the facet points directly at the Sun;
- $0 < \mu_{0,i} < 1$: sunlight strikes it obliquely;
- $\mu_{0,i} = 0$: it lies on the day-night boundary, the terminator;
- $\mu_{0,i} < 0$: it faces away from the Sun and belongs to the night side.

The sign of a dot product is a fast front-facing test



8.1 Where the illumination test is used in the code

`observation()` first computes all Sun-facing dot products at once:

```
sun_dot = np.tensordot(shape.normals, subsolar, axes=(1, 1))
lit = sun_dot > 0
sun_dot = sun_dot * lit
```

If there are N_f facets and N_t times, `sun_dot` has shape

$$N_f \times (N_t + 1).$$

The extra column is the reference geometry discussed later. Multiplication by the Boolean array changes every negative or zero entry to zero.

8.2 Small numerical example

Suppose

$$\hat{\mathbf{n}}_i = (0.8, 0.6, 0), \quad \hat{\mathbf{s}} = (1, 0, 0).$$

Then

$$\mu_{0,i} = 0.8(1) + 0.6(0) + 0(0) = 0.8 > 0.$$

The facet is illuminated.

If the Sun direction changes to $(-1, 0, 0)$, then

$$\mu_{0,i} = -0.8 < 0,$$

so the facet is rejected as unilluminated.

9. Visibility test: can the observer see this facet?

Let $\hat{\mathbf{o}}(t)$ be the unit vector from the asteroid toward the observer. Define the emission cosine

$$\mu_i(t) = \hat{\mathbf{n}}_i \cdot \hat{\mathbf{o}}(t).$$

For unit vectors,

$$\mu_i = \cos e_i,$$

where e_i is the emission angle.

The visibility decision is

$$\text{facet is visible} \iff \mu_i > 0.$$

Interpretation:

- $\mu_i = 1$: face-on to the telescope;
- $0 < \mu_i < 1$: visible at an oblique angle;
- $\mu_i = 0$: exactly on the apparent limb;
- $\mu_i < 0$: on the far side.

The code is:

```
obs_dot = np.tensordot(shape.normals, subobserver, axes=(1, 1))
vis = obs_dot > 0
obs_dot = obs_dot * vis
```

10. Why both tests are necessary

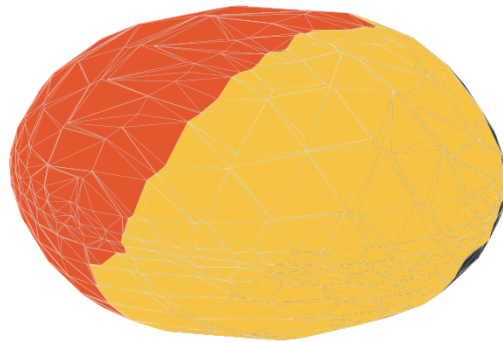
A facet contributes reflected sunlight only if it is both illuminated and visible:

$$I_i(t) = \begin{cases} 1, & \mu_{0,i} > 0 \text{ and } \mu_i > 0, \\ 0, & \text{otherwise.} \end{cases}$$

Four cases exist:

Illumination	Visibility	Physical meaning	Flux contribution
No	No	night side and far side	zero
Yes	No	sunlit but hidden behind asteroid	zero
No	Yes	visible night side	zero
Yes	Yes	sunlit surface visible from Earth	included

Facet selection on Juno at one synthetic observing time



 neither	 visible only
 lit only	 lit and visible: contributes flux

In the illustrated example, orange facets are the only facets that can contribute reflected flux. Yellow facets receive sunlight but cannot be seen. Blue facets are visible but dark. Dark-grey facets satisfy neither test.

11. Why dot products work only because the model is convex

The dot-product tests determine whether a plane faces a direction. They do not check whether another triangle lies between that plane and the Sun or observer.

For a convex body, this is sufficient:

- a facet with $\mu_{0,i} > 0$ cannot be shadowed by another part of the convex body;

- a facet with $\mu_i > 0$ cannot be hidden behind another part of the convex body.

For a non-convex body, positive dot products are necessary but not sufficient. Ray tracing would add tests such as

$$\mathbf{r}_{\text{Sun}}(q) = \mathbf{q}_i + \epsilon \hat{\mathbf{n}}_i + q \hat{\mathbf{s}}, \quad q > 0,$$

and ask whether that ray intersects any other facet. A similar ray toward $\hat{\mathbf{o}}$ tests occlusion. The small offset ϵ prevents the ray from immediately intersecting its own triangle.

The current Python code performs no ray-triangle intersection. Therefore, use it with a convex DAMIT model unless you intentionally add a non-convex renderer.

12. The scattering law: how much light comes from an accepted facet?

The two tests decide whether a contribution is permitted. They do not yet determine its strength.

The code uses an LS/Lambert mixture:

$$S_i(\mu_{0,i}, \mu_i) = \mu_i \mu_{0,i} \left(0.1 + \frac{1}{\mu_i + \mu_{0,i}} \right).$$

In code:

```
lsl = obs_dot * sun_dot * (0.1 + 1 / (obs_dot + sun_dot))
```

The terms have distinct roles:

- $\mu_{0,i}$ decreases the contribution for grazing sunlight;
- μ_i decreases the apparent projected area near the observer's limb;
- $1/(\mu_i + \mu_{0,i})$ supplies a Lommel-Seeliger-like component;
- 0.1 supplies a smaller Lambert-like component.

When a facet is rejected, both the mask and `nan_to_num()` ensure its scattering weight becomes zero.

12.1 Example

For a well-oriented facet with

$$\mu_{0,i} = 0.8, \quad \mu_i = 0.6,$$

the weight is

$$S_i = (0.6)(0.8) \left(0.1 + \frac{1}{1.4} \right) \approx 0.3909.$$

For a night-side facet, the masked $\mu_{0,i}$ is zero and therefore $S_i = 0$.

13. Area and reflectance weighting

For a uniformly reflective surface, the flux contributed by facet i is proportional to

$$F_i(t) = A_i S_i(t).$$

For a spatially varying reflectance map, it becomes

$$F_{i,B}(t) = A_i R_{i,B} S_i(t),$$

where $R_{i,B}$ is the reflectance of facet i in wavelength band B .

The supplied `observation()` function currently implements

```
flux = areas * lsl
```

which is equivalent to setting

$$R_{i,B} = 1$$

for every facet. It therefore generates a **uniform-colour/null-model light curve**, not a spotted surface by itself.

This distinction is important. To simulate a colour spot, a reflectance factor must be applied to selected facets, and that factor must differ between bands.

13.1 Example of a synthetic red spot

If a selected surface region is ordinary in band 1 but more reflective in band 2, one might assign

$$R_{i,1} = 1.0,$$

$$R_{i,2} = 1.5$$

inside the spot, while keeping $R = 1$ elsewhere. The total flux in the two bands then changes differently whenever that region is illuminated and visible.

14. Summing the unresolved asteroid

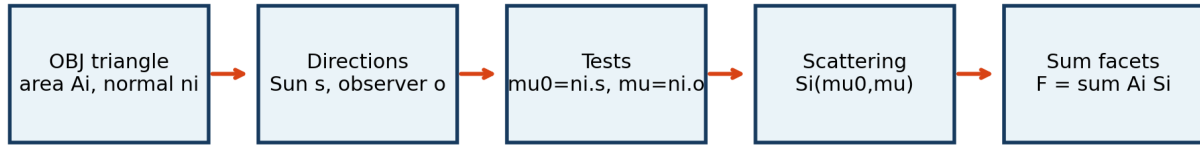
The telescope does not retain a separate measurement for every triangle. The code sums all accepted facets:

$$F_{\text{shape}}(t) = \sum_{i=1}^{N_f} A_i S_i(t)$$

or, with reflectance,

$$F_B(t) = \sum_{i=1}^{N_f} A_i R_{i,B} S_i(t).$$

This is the disk-integrated flux of an unresolved asteroid.



Repeat for every observation time, then convert total flux to magnitude

15. Why the code appends `[1, 0, 0]`

The following lines add one artificial reference geometry:

```
subsolar = np.append(np.transpose(sun_pos), [[1, 0, 0]], axis=0)
subobserver = np.append(np.transpose(obs_pos), [[1, 0, 0]], axis=0)
```

At the reference geometry, both the Sun and observer lie along the positive asteroid x direction. Therefore:

- phase angle is effectively zero;
- the subsolar and subobserver points coincide;
- the calculated final element `sum_flux[-1]` provides a normalization flux.

The shape-induced relative flux is

$$f(t) = \frac{F_{\text{shape}}(t)}{F_{\text{ref}}}.$$

This normalization removes the arbitrary overall scale of the OBJ coordinates.

16. Flux-to-magnitude conversion

Astronomical magnitude differences are logarithmic:

$$\Delta m = -2.5 \log_{10} \left(\frac{F}{F_{\text{ref}}} \right).$$

Consequently, a larger flux gives a smaller magnitude. “Brighter” means numerically lower magnitude.

The code calculates the shape/rotation term as

```
-2.5 * np.log10(sum_flux[:-1] / sum_flux[-1])
```

16.1 Distance correction

Reflected flux decreases approximately as

$$F \propto \frac{1}{r^2 \Delta^2},$$

where r is the Sun-asteroid distance and Δ is the observer-asteroid distance. Converting this factor to magnitudes gives

$$5 \log_{10}(r\Delta).$$

16.2 Phase function

Brightness also changes with phase angle α . The supplied example uses an H, G phase function:

```
juno_phasefunc = ph.models.HG(H=5.19, G=0.320)
```

The model magnitude is therefore conceptually

$$m_B(t) = -2.5 \log_{10} \left(\frac{F_{\text{shape}, B}(t)}{F_{\text{ref}, B}} \right) + \Phi_B[\alpha(t)] + 5 \log_{10}[r(t)\Delta(t)].$$

The code's `phase_function.eval(eph['phase'])` supplies the phase-function magnitude term, including its adopted absolute-magnitude convention.

17. Array dimensions inside observation()

Suppose:

$$N_f = 2036, \quad N_t = 1000.$$

Then the main arrays have these approximate shapes:

Variable	Shape	Meaning
<code>shape.normals</code>	(2036, 3)	normal of each facet
<code>sun_pos</code>	(3, 1000)	Sun direction at each time
<code>obs_pos</code>	(3, 1000)	observer direction at each time
<code>subsolar</code>	(1001, 3)	times plus reference geometry
<code>sun_dot</code>	(2036, 1001)	μ_0 for every facet and time
<code>obs_dot</code>	(2036, 1001)	μ for every facet and time
<code>lsl</code>	(2036, 1001)	scattering weight
<code>flux</code>	(2036, 1001)	area-weighted facet flux
<code>sum_flux</code>	(1001,)	total flux at each time plus reference

`np.tensordot()` is used to calculate millions of dot products without a Python loop.

18. Where this code belongs in your project

The complete project has four conceptual stages.

Stage A - Predict geometry

Inputs:

- light-time-corrected observation epochs;
- observatory location;
- asteroid orbit or name.

Function:

```
juno_ephemeris = kete_ephem(times, "Palomar Mountain", name="Juno")
```

Output: Sun/observer geometry, phase angle, distances, RA/Dec and elongation.

Stage B - Predict uniform-surface brightness

Inputs:

- juno.obj;
- DAMIT pole and period;
- phase-function parameters;
- ephemeris from Stage A.

Functions:

```
juno_shape = obj3d_import("juno.obj")
juno_model = observation(
    juno_ephemeris,
    juno_rotation,
    juno_shape,
    phase_function=juno_phasefunc
)
```

Output: model magnitude and asteroid-fixed Sun/observer directions for every epoch.

This is where the illumination and visibility tests are used. They are internal calculations required to build each predicted magnitude.

Stage C - Compare real photometry with the model

For real ZTF magnitude $m_{B,\text{obs}}(t)$ and model prediction $m_{B,\text{mod}}(t)$, calculate

$$\Delta m_B(t) = m_{B,\text{obs}}(t) - m_{B,\text{mod}}(t).$$

The residual is not calculated facet by facet. It is calculated once per real observation after all facet contributions have been summed into the model magnitude.

Stage D - Apply the colour-variation statistic

```
zscore = spot_detection(real_observations, uniform_model)
```

This stage asks whether residuals depend differently on viewing geometry in different filters. The triangular renderer supplies the trustworthy uniform-surface prediction needed by the test.

19. What the two major code sections are for

Code section	Scientific purpose	When you use it
obj3d and obj3d_import()	read and prepare the DAMIT mesh	once for each asteroid model
kete_ephem()	calculate orbital/observing geometry	at all ZTF observation times
asterocentric()	rotate directions into body frame	internally for every observation
observation()	render the uniform-surface light curve	to predict magnitudes and shapes
spot_detection()	compare band-dependent residual correlations	after real and model tables are built
final Juno block	demonstration	initial learning and validation

20. What the supplied demonstration currently does

The final block:

1. imports the Juno mesh;
2. defines Juno's spin pole and period;
3. defines an H, G phase function;
4. creates 1,000 evenly spaced times over three days;
5. retrieves Juno's ephemeris for Palomar Mountain;
6. predicts its magnitude at every time;
7. plots a model light curve.

It does **not** yet:

- download ZTF observations;
- match ZTF epochs and filters;
- fit model parameters to ZTF data;
- create a non-uniform spot simulation;
- call `spot_detection()`;
- produce a probability for real Juno data.

It is a forward-model demonstration, not the complete research pipeline.

21. Exact walkthrough of `spot_detection()`

This function is downstream of the shape calculation.

21.1 Residual vector

```
y = obs['magnitude'] - mod['magnitude']
```

Mathematically:

$$y_j = \Delta m_j = m_{\text{obs},j} - m_{\text{mod},j}.$$

21.2 Average illumination-viewing direction

```
X = (sub_sun + sub_observer) / 2
```

For observation j :

$$\tilde{\mathbf{E}}_j = \frac{1}{2} (\hat{\mathbf{s}}_j + \hat{\mathbf{o}}_j).$$

This is a rough direction toward the surface region that is jointly illuminated and visible.

21.3 Regression direction

The code fits

$$\Delta m_j \approx k_x \tilde{E}_{x,j} + k_y \tilde{E}_{y,j} + k_z \tilde{E}_{z,j}.$$

The fitted vector

$$\mathbf{k} = (k_x, k_y, k_z)$$

selects the direction along which residuals change most strongly in this linear approximation.

21.4 Spearman correlation in each band

For each filter B , the code calculates

$$\rho_B = \text{corr}_{\text{Spearman}}(\mathbf{X}_B \mathbf{k}, \Delta m_B).$$

21.5 Fisher difference score

Each correlation is transformed using

$$z_B = \text{atanh}(\rho_B) = \frac{1}{2} \ln \left(\frac{1 + \rho_B}{1 - \rho_B} \right).$$

For the largest and smallest band correlations, the code computes

$$z_{\text{mathrmscore}} = \frac{z_{\max} - z_{\min}}{\sqrt{\frac{1}{n_{\max}-3} + \frac{1}{n_{\min}-3}}}.$$

A large positive score means the geometry-residual relationship differs strongly between filters, which is evidence against a uniformly coloured surface under the model assumptions.

22. Important distinction: illumination test versus statistical test

These are different tests at different levels.

Illumination test: It takes one facet normal and the Sun direction. It asks whether that facet is on the lit side and returns the incidence cosine μ_0 together with a true/false decision.

Visibility test: It takes one facet normal and the observer direction. It asks whether the observer can see that facet and returns the emission cosine μ together with a true/false decision.

Surface-colour statistical test: It takes all observed-minus-model residuals in multiple filters. It asks whether different filters depend differently on surface geometry and returns a Fisher z score.

The first two are deterministic geometry calculations. The final one is a statistical inference calculation.

23. Issues to correct before running the supplied code

The scientific structure is clear, but the supplied file needs a few practical corrections.

23.1 Missing stats import

`spot_detection()` calls `stats.spearmanr`, but the file does not import `stats`. Add:

```
from scipy import stats
```

23.2 Undefined plot

The function checks if `plot:`, but `plot` is neither a parameter nor defined globally. Prefer:

```
def spot_detection(obs, mod, plot=False):
```

23.3 Working directory for `juno.obj`

This line expects the OBJ file in the process's current working directory:

```
juno_shape = obj3d_import('juno.obj')
```

Running the script from a different directory will cause `FileNotFoundError`. Use a path based on the script location.

23.4 Required packages

The script imports NumPy, Matplotlib, Astropy, SciPy, pandas, scikit-learn, kete and phunk. kete and phunk may require separate installation. The mesh-reading and facet mathematics do not depend on ZTF, but ephemerides require kete, and the H , G example requires phunk.

23.5 Inputs must be aligned

`obs` and `mod` must refer to the same observations in the same row order. Every row should agree in:

- epoch;
- filter;
- asteroid;
- light-time convention.

Otherwise, the residuals are meaningless even if the code runs.

23.6 Two unused arrays

`X1` and `X2` are created in `spot_detection()` but not used later. They are harmless, but they can be removed unless retained for debugging or plotting.

24. How to use the code for one real ZTF asteroid

The recommended order is:

1. Select one asteroid with a reliable convex DAMIT model and spin solution.
2. Download its ZTF photometry with epochs, filters, magnitudes and uncertainties.
3. Clean obvious flags and duplicate measurements.
4. Convert timestamps consistently and apply the required light-time handling.
5. Evaluate the ephemeris at the exact observation epochs.
6. Import the corresponding OBJ model.
7. insert the DAMIT pole, period, reference epoch and rotational phase convention.
8. Fit or adopt a phase function separately for each filter as appropriate.
9. Run `observation()` at the exact ZTF epochs to obtain uniform-surface predicted magnitudes.
10. Fit any necessary band zero points without erasing rotational structure.
11. Form observed-minus-model residuals.
12. Run the statistical test.
13. Use simulations and uncertainty perturbations to calibrate false-positive behaviour.
14. Inspect whether a signal survives plausible shape, pole, period, phase-function and photometric errors.

The illumination and visibility code is used in step 9. You do not manually apply it to the ZTF table before calling the renderer.

25. How a colour spot should enter a simulation

The present renderer assumes equal reflectance. A spot simulation needs a surface-selection rule and a band-dependent reflectance.

For a spot centred on unit direction $\hat{\mathbf{p}}$ with angular radius ϕ_{spot} , select facet i when

$$\arccos \left(\frac{\mathbf{q}_i}{\|\mathbf{q}_i\|} \cdot \hat{\mathbf{p}} \right) \leq \phi_{\text{spot}}.$$

Equivalently, avoid the expensive inverse cosine and test

$$\frac{\mathbf{q}_i}{\|\mathbf{q}_i\|} \cdot \hat{\mathbf{p}} \geq \cos \phi_{\text{spot}}.$$

Then assign $R_{i,B}$ and use

$$F_B(t) = \sum_i A_i R_{i,B} S_i(t).$$

For the null model, keep $R_{i,B}$ spatially constant within each band. For the spotted synthetic data, change $R_{i,B}$ within the selected region in one or both bands.

26. Validation tests you should perform first

Before real-data analysis, verify the renderer with small controlled tests.

26.1 Normal-length test

Require

$$\|\hat{\mathbf{n}}_i\| \approx 1$$

for every facet.

26.2 Outward-normal test

For a body centred near $\mathbf{c}$, most or all facets should satisfy

$$\hat{\mathbf{n}}_i \cdot (\mathbf{q}_i - \mathbf{c}) > 0.$$

All supplied Juno facets pass this test.

26.3 Closed-mesh test

Every undirected edge should normally be used by two triangular facets. The Euler check should be consistent with the expected topology.

26.4 Opposition symmetry test

Set Sun and observer directions equal. Only the hemisphere facing that direction should contribute.

26.5 Reversal test

If both directions are reversed, the opposite hemisphere should contribute.

26.6 Rotation periodicity test

With fixed external geometry,

$$m(t + T) \approx m(t).$$

26.7 Sphere test

A sufficiently fine uniform sphere at fixed phase angle should have little or no rotational light-curve variation.

26.8 Scale test

Multiplying every OBJ coordinate by the same factor changes all facet areas by that factor squared. Because the code normalizes by a reference flux from the same scaled mesh, the relative magnitude curve should remain unchanged.

27. What can create a false colour signal

A residual pattern is evidence only relative to the assumed model. It can also be produced by:

- an incorrect rotation period, causing rotational phase drift;
- a wrong pole solution;
- a DAMIT shape that does not fit the new viewing geometries;
- filter-dependent phase-function error;
- incorrect light-time handling;
- mismatched observation/model rows;
- calibration offsets or colour terms;
- sparse and unequal sampling between filters;
- unremoved bad photometry;
- genuinely non-convex shadowing not captured by the convex model.

This is why Monte Carlo perturbations and validation against synthetic null data are required before interpreting a large z score physically.

28. The complete scientific chain in one place

For each real observation j :

1. Obtain reception time t_j and correct the relevant rotational epoch for light travel.
2. Compute asteroid-to-Sun direction $\hat{\mathbf{s}}_j$.
3. Compute asteroid-to-observer direction $\hat{\mathbf{o}}_j$.
4. Rotate both into the asteroid-fixed frame.
5. For every facet i , compute

$$\mu_{0,ij} = \hat{\mathbf{n}}_i \cdot \hat{\mathbf{s}}_j, \quad \mu_{ij} = \hat{\mathbf{n}}_i \cdot \hat{\mathbf{o}}_j.$$

6. Set the facet contribution to zero unless both values are positive.
7. For accepted facets compute

$$S_{ij} = \mu_{ij}\mu_{0,ij} \left(0.1 + \frac{1}{\mu_{ij} + \mu_{0,ij}} \right).$$

8. Sum

$$F_{B,j} = \sum_i A_i R_{i,B} S_{ij}.$$

9. Convert to model magnitude and include phase and distance terms.
10. Compute residual

$$\Delta m_{B,j} = m_{B,\text{obs},j} - m_{B,\text{mod},j}.$$

11. Relate residuals to the average Sun-observer direction.
12. Calculate one correlation per filter.
13. Compare filter correlations with the Fisher statistic.
14. Calibrate detection and false-positive rates using simulated datasets and perturbed model parameters.

29. Plain-language explanation for a professor

I use a DAMIT model that represents an asteroid as thousands of small triangles. At every observation time, I rotate the directions of the Sun and Earth into the asteroid's own coordinate system. For each triangle, I check whether it faces the Sun and whether it faces Earth. Only triangles passing both checks reflect light that can reach the telescope. I calculate their reflected-light weights, multiply by their areas, and add them to predict the asteroid's total brightness. This gives the brightness expected from a uniformly coloured surface. I compare that prediction with real measurements in different filters. If the remaining brightness errors repeatedly depend on which surface region is visible, and the relationship differs significantly between filters, it may indicate surface-colour variation. Simulations are then used to determine how often the same result could appear because of noise or model errors.

30. Formula reference

Purpose	Formula
Edge vectors	$\mathbf{a}_i = \mathbf{v}_{i1} - \mathbf{v}_{i3}, \mathbf{b}_i = \mathbf{v}_{i2} - \mathbf{v}_{i1}$
Cross product	$\mathbf{c}_i = \mathbf{a}_i \times \mathbf{b}_i$
Facet area	$A_i = \frac{1}{2} \ \mathbf{c}_i\ $
Unit normal	$\hat{\mathbf{n}}_i = \mathbf{c}_i / \ \mathbf{c}_i\ $
Centroid	$\mathbf{q}_i = (\mathbf{v}_{i1} + \mathbf{v}_{i2} + \mathbf{v}_{i3})/3$
Illumination cosine	$\mu_{0,i} = \hat{\mathbf{n}}_i \cdot \hat{\mathbf{s}}$
Visibility cosine	$\mu_i = \hat{\mathbf{n}}_i \cdot \hat{\mathbf{o}}$
Facet accepted	$\mu_{0,i} > 0 \text{ and } \mu_i > 0$
Scattering weight	$S_i = \mu_i \mu_{0,i} [0.1 + 1/(\mu_i + \mu_{0,i})]$
Facet flux	$F_{i,B} = A_i R_{i,B} S_i$
Total flux	$F_B = \sum_i F_{i,B}$
Relative magnitude	$\Delta m = -2.5 \log_{10}(F/F_{\text{ref}})$
Distance term	$5 \log_{10}(r\Delta)$
Residual	$m_{B,\text{obs}} - m_{B,\text{mod}}$

Purpose	Formula
Fisher transform	$z_B = \text{atanh}(\rho_B)$
Band-difference statistic	$(z_1 - z_2) / \sqrt{1/(n_1 - 3) + 1/(n_2 - 3)}$

References and software

- Humes, O. and Agarwal, J. (2026), Prospects for detecting surface color heterogeneity on asteroid surfaces from sparse multiband photometric survey data, *Astronomy & Astrophysics* 711, A209.
- DAMIT asteroid models and `lcgenerator`: <https://damit.cuni.cz/>
- kete documentation: <https://dahlend.github.io/kete/>
- phunk documentation: <https://phunk.readthedocs.io/>
- Fink/ZTF Solar System search service: https://doc.ztf.fink-broker.org/services/search/solar_system/

The numerical mesh counts and diagnostic statements in this report were calculated directly from the supplied `juno.obj`. Code references were traced directly from the supplied `example_code.py`.